The linear stage after F4, end to end, on a corpus of Gröbner bases

2026-10-27 · `research/postgb_linear_stage_20261026` · protocol frozen in `544eb84f95` before any measured session; amendments 1 (`4d55b7306e`, harness F4-budget bug, every card re-run as R2), 2 (`42bb4d54fb`, host change) and 3 (`b8b4a97983`, the fast arm's counter fixed and every count re-measured as R3; the negation-map rho reference). 253 systems in five families, 7 interleaved rounds per system, single thread. Every arm agreed on every system.

Question

The owner asked for the linear-algebra stage to be sped up until it takes the leaderboard's linear-algebra section. The measurement is end to end for the linear stage alone, not the whole index calculus. It runs on a variety of systems whose Gröbner bases come out of F4.

The linear stage takes a monic grevlex basis (`f4_fp`) to every F_p-rational point, each checked against the input: staircase, normal forms, M_{x_n}, minimal polynomial, roots, coordinates and the check.

Arms

- **dense:** all M_i, Hessenberg characteristic polynomial, a null space per root.
- **sparse (the incumbent,** the leader of the IC leaderboard section "Linear stage after F4 (sparse FGLM)"**):** Wiedemann/ Berlekamp–Massey on M_{x_n}, then shift-chain eliminations per root.
- **fast (new, `zero_dim_la::fast_stage`):**
  - M_{x_n} as hybrid dense/sparse columns;
  - one transposed Krylov pass $(M^T)^j u$ for every coordinate;
  - Berlekamp–Massey;
  - roots from $\gcd(f, x^p - x)$, with Karatsuba/Barrett powering;
  - coordinates read out by the rational univariate representation;
  - lazy u32/u64 arithmetic with AVX-512/AVX2 kernels chosen at run time;
  - falls back to dense inside its own clock.

Result

prediction	statement	outcome
P1	every family sparse/fast ≥ 1.5, interval above 1	**held**: 7.1–45.5×
P2	in `dense`, sparse/fast grows with δ	**held**: Spearman ρ = 0.93 (153 systems)
P3	fast gives up on ≤ 5%	**held**: 1 of 253 (`gaudry-d4 p97-4`, x_n not separating)
P4	DLP logs correct; counted muls within ±25% on every system; S within ±10%; lower linear-stage wall time	**failed** on the counted clause (R3: 0.77–2.14; outside on 121 systems); the other three held
P5	portable fast (no vector kernels) still ≥ 1.2	**held**: 6.3–11.1×

The linear stage is 7–46× faster than the incumbent in every family (host B)

family	systems	δ	sparse ms	fast ms	sparse ÷ fast [95%]	dense ÷ fast	counted sparse ÷ fast
jv-nagao	48	64	3.05	0.40	**7.60 [7.43, 7.78]**	17.9	0.98–1.32
gaudry-s4	24	64	2.21	0.32	**7.13 [6.80, 7.49]**	14.6	1.12–1.54
gaudry-d4	24	512	313	16.5	**17.8 [13.7, 20.5]**	109	0.90–1.56
gaudry-s5	4	4096	229,741	4,849	**45.5 [42.8, 47.2]**	—	0.77–0.89
dense	153	32–1024	104	4.64	**20.0 [18.3, 21.7]**	60.6	0.99–2.14

The ms columns give each family's median system. The ratio is the geometric mean of the per-system median wall-time ratios, with a bootstrap interval over systems.

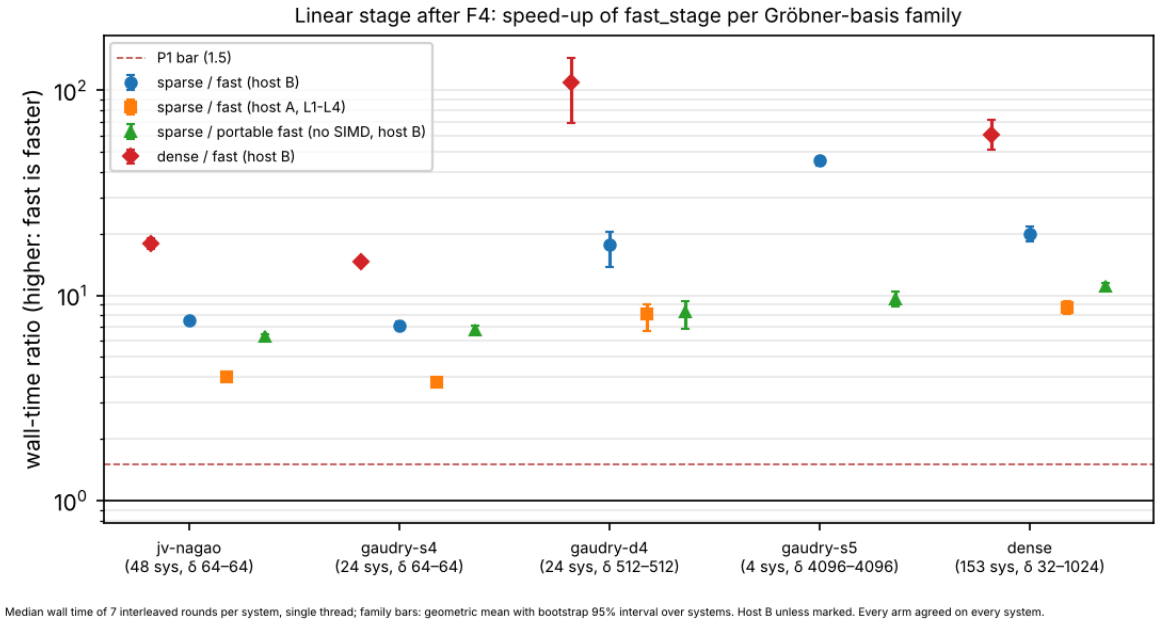


Figure 1: Per-family speed-up. The host-A bars are the first runs L1–L4 on the other host. The triangles are the portable ablation.

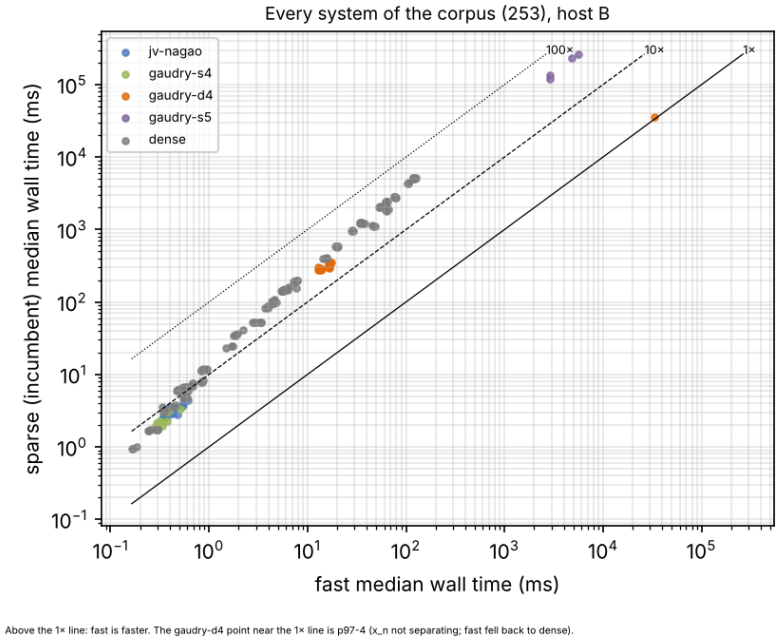


Figure 2: Every system: sparse against fast median wall time.

Where the gain comes from. It grows with δ (Spearman ρ = 0.93), and most of it is in the cost per multiplication.

- **Counted multiplications:** these are the fixed counts of amendment 3, R3-C.
  - `fast` does fewer on four families: geometric means of 1.13, 1.26, 1.05 and 1.42.
  - It does more on `gaudry-s5` (0.83).
  - P4′s preregistered ±25% fails: the range is 0.77–2.14.
- **Vector kernels:** they are worth 1.0–4.0× (portable ÷ vector `fast`: 1.15, 1.04, 2.16, 4.01, 1.83). The remaining 6–11× is the algorithm and the arithmetic layout.

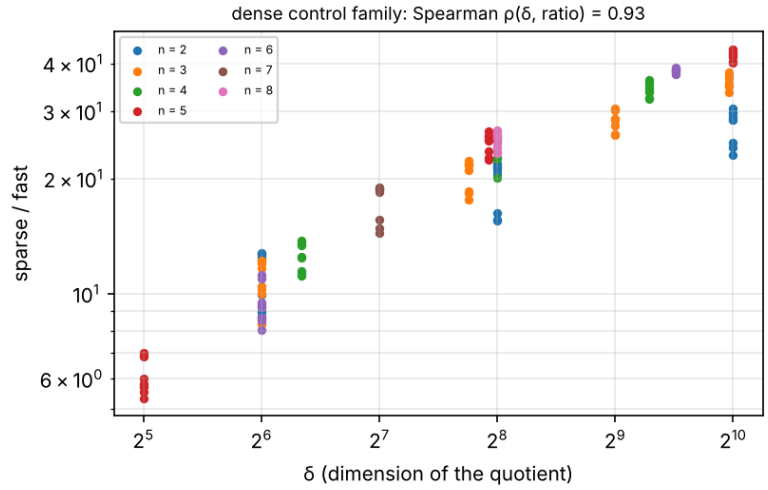


Figure 3: `dense` control family: speed-up against δ.

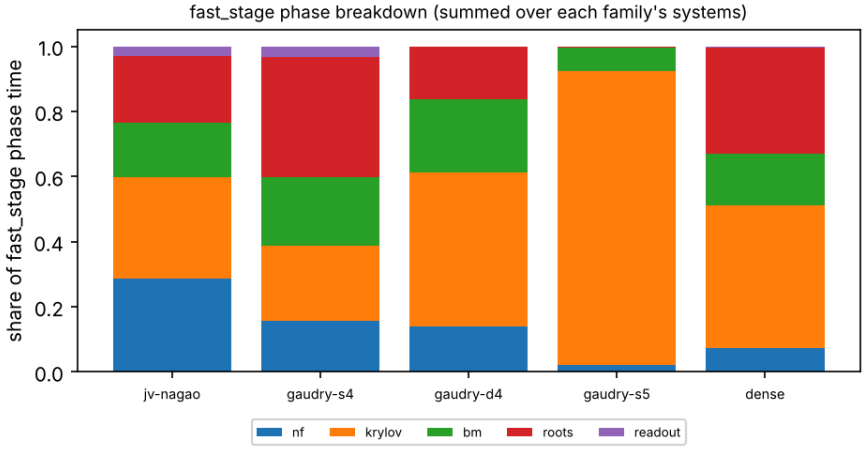


Figure 4: Where `fast` spends its time.

- The Krylov pass takes 23–90% (90% at δ = 4096).
- The root step takes 16–37%.
- The normal forms take up to 29%.

Inside the index calculus: `jv_cover` DLP, end to end

R2-D1 and R2-D2 ran the same command with the same seeds and relations, `JV_LIN=sparse` against `JV_LIN=fast`; they give the wall time. R3-D1 and R3-D2 re-ran it with the fixed counter, which gives S, and with the negation-map rho in the same process, which is the matched reference. Every log was correct.

p	seed	rel.	lin. stage s	lin ÷	S	S/rho	S/rho$_{\text{neg}}$ (distance)
53	1	31	64.8 ⟶ 12.8	**5.07**×	731.6 ⟶ 727.1	664 ⟶ 660	1319 ⟶ **1310**
53	2	33	29.5 ⟶ 6.2	**4.79**×	328.8 ⟶ 326.5	286 ⟶ 284	510 ⟶ **506**
101	1	53	162.0 ⟶ 22.6	**7.16**×	224.4 ⟶ 220.4	151 ⟶ 149	326 ⟶ **320**
101	2	50	220.7 ⟶ 36.5	**6.05**×	365.5 ⟶ 359.0	293 ⟶ 287	262 ⟶ **257**

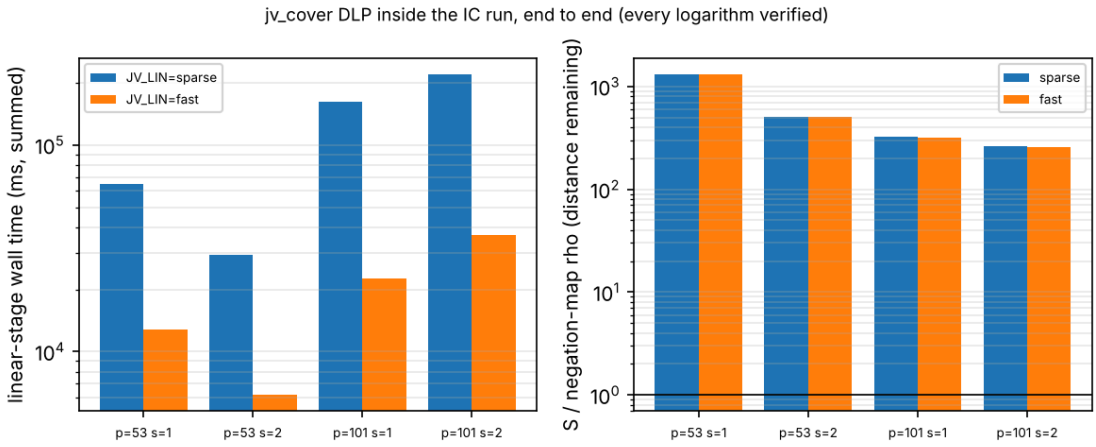


Figure 5: `jv_cover` DLP: linear-stage wall time and S/rho.

Linear stage. Its wall time inside the IC run falls 4.8–7.2×. The whole run is 27–37% shorter at p = 101 and unchanged at p = 53, where the same-process rho dominates.

Counted S falls 0.6–1.8%. `fast` does 3–8% fewer linear-stage multiplications. Under R2′s formula counter, S had appeared to rise about 3%.

Tables A–C. No row moves.

§5a band. `jv_cover` emits no unpriced native work, so S is the same at 0, 0.1 and 1 GAE per operation. Rho is cheaper than both arms across the band. Against the negation-map rho, S/rho with `fast` is 257–1310; that is the distance remaining.

Host dependence

The worker moved hosts between L4 and the R2 cards:

- host A: Xeon 2.1 GHz, 8 MiB L2, 260 MiB L3;
- host B: Xeon 2.8 GHz, 4 MiB L2, 33 MiB L3.

Hosts are not pooled. On host A, sparse ÷ fast was 4.01, 3.79, 8.14 and 8.75 for jv-nagao, gaudry-s4, gaudry-d4 and dense. The incumbent slows about 3× on B's smaller caches, `fast` about 1.3×. Both hosts clear P1 by a wide margin.

Decision

P1 and the correctness gate hold, so `fast` becomes the default linear stage:

- `JV_LIN` defaults to `fast`, and the generic entry point `solve_zero_dim` follows it;
- `JV_LIN=sparse` keeps the incumbent.

It leads a new leaderboard component section, "Linear stage after F4 across Gröbner-basis corpora". The class is engineering, a component speed-up.

Follow-up. The one give-up, `p97-4`, is fixed by a random separating linear form instead of x_n. That is outside this protocol.

Deviations

1. Card L1′s first attempt was refused by its disk check (588 MB free, below 1 GB), and is kept under `cards/`.
2. A harness F4-budget bug skipped 11 systems in L3 and L4. Amendment 1 fixed it, and all ten cards were re-run as R2.
3. The host changed after L4. Amendment 2 reports the hosts separately, raises the timeouts to 7000 s, and has the main session run the long cards.
4. Conductor was unreachable all session (its hooks let edits through).
5. No `CHECKPOINT_UPLOAD_TOKEN` was set, so the S3 resume checkpoint could not be pushed.
6. A PR review found the fast arm's root powering charged by a formula (amendment 3). The counter was fixed and every count re-measured in R3-C, R3-D1 and R3-D2; wall time is R2′s. The analysis moved from Python to Rust.